Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Ahmed Gouda Ismail[1*], Mohamed S. Mohamed[1] and Tarek Ahmed Mahmoud[2]

[1*]Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.

[2]Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com
Contributing authors: mohamedms@mtc.edu.eg; tarek.mohamed@eui.edu.eg

Abstract

Many multi-object tracking systems combine a detector and a tracker developed separately. When the detector is replaced, retrained, or quantized for embedded hardware, its confidence scores change, but the tracker keeps thresholds set for another detector. This paper presents a training-free self-calibrating control layer (SCL) that sits between a frozen detector and a frozen tracker. While running, it splits the detector scores of recent frames with Otsu's histogram threshold and decides whether they are clean or noisy. On clean scores it keeps the tracker's settings; on noisy scores it chooses which candidates reach the tracker, assigns them new scores, and sets the tracker's thresholds. It also adds a continuation step for trackers that lack one. It needs no labels and no future frames. One frozen configuration was tested on seven published trackers, four detection sources, and two datasets. Relative to the tracker alone, it raised the higher order tracking accuracy (HOTA) of two single-stage trackers by 0.9%: OC-SORT, used during development, and PD-SORT, which was not. When all detector scores were halved, two trackers failed completely; with SCL, both recovered at least 97% of their HOTA on the original scores. With a noisy RT-DETR-L detector, it raised the multiple object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 12.6% and 16.3%. On four trackers with a low-score step and well-matched detections, HOTA changed by less than 0.1%. Paired bootstrap analysis supports these results. SCL takes 2.95 ms per frame on a central processing unit, which suits real-time use.

Keywords: Multi-object tracking, Tracking by detection, Detector confidence calibration, Online adaptation, Otsu thresholding, Real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video and gives each object the same identity over time. It is used in surveillance, driving, and drone systems [1–3], and many of these systems must give results with little delay. Most trackers follow the tracking-by-detection design. A detector gives a score to each candidate box in a frame, and the tracker links the boxes whose scores pass its thresholds into tracks [4–6].

In real systems, the detector and the tracker are rarely built together. A detector is replaced by a faster model, retrained for a new camera, or made smaller (quantized) for embedded hardware. The tracker, however, keeps the thresholds that its

authors set for another detector. Detector scores are not calibrated across models and conditions [7], so a new detector usually gives scores on a different scale, and nothing in the system notices the change.

The effect can be large. In this paper, halving the scores of a published detector reduces two well-known trackers to 0 higher order tracking accuracy (HOTA) [8], because no detection passes their fixed thresholds (Sect. 7.3). Retuning every tracker for every detector needs labelled data from the new setting, which a working system usually does not have.

Current methods solve only part of this problem. Trackers with adaptive thresholds change their own thresholds from the detections of each frame, but each method is built into one tracker [9–11]. Score calibration methods correct the detector scores, but they need labelled data [7]. Adaptive inference methods learn or measure a policy for one model [12, 13]. What is missing is a component that needs no training, works outside the tracker, adjusts itself from the detector's own scores, and leaves the tracker alone when it already fits those scores.

This paper presents such a component: a self-calibrating control layer (SCL) that sits between a fixed (frozen) detector and a fixed tracker. It reads the detector scores and the settings that the tracker declares, and it decides while running whether the tracker needs help. The main idea is simple: the detector's own scores show whether its output fits the tracker. SCL looks at the scores of recent frames and checks how clearly they separate confident objects from clutter. If the separation is clear, the tracker keeps its own settings, and the layer only adds help that the tracker lacks, such as a step that continues tracks. If it is not clear, SCL chooses which candidates reach the tracker, assigns them new scores, and sets the tracker's thresholds. The novelty is therefore a layer outside the tracker that needs no labels, uses one fixed configuration for many trackers, and stays out of the way when the tracker already fits the detector.

The main contributions are:

1. SCL, a control layer that needs no training, adjusts itself from the detector scores, uses no future frames, keeps the tracker's settings when the scores are clean, and takes full control only when they are noisy (Sect. 4).
2. A host contract, a set of four numbers from the tracker's published settings, that lets one fixed configuration work with single-stage and two-stage trackers without changing them (Sect. 4.5).
3. A development path from always-on banding (AOB), an earlier design that always intervened, in which each step is compared with that baseline (Sect. 5).
4. A test of one fixed configuration on seven published trackers, four detection sources, and two datasets, with development and predeclared external groups, paired bootstrap intervals, and a speed measurement on a central processing unit (CPU) (Sects. 6 and 7).

The rest of the paper is organized as follows. Section 2 describes the data, the evaluation roles, and the metrics, and Section 3 reviews related work. Section 4 explains SCL and all its settings, and Section 5 shows how it was developed from an earlier design that always intervened. The configuration was then frozen before any external system was run. Section 6 gives the experimental setup, Section 7 the results and their analysis, and Section 8 concludes the paper.

2 Data and Evaluation

2.1 Detectors and data

Table 1 lists the evaluation cells. A cell is one combination of data, detector, and tracker (the trackers are introduced in Sect. 6). On MOT17 val-half [14] the trackers use the published YOLOX-X detections [15]. The emission floor is the lowest detector score that is passed on; it is 0.01 or 0.1 (Sect. 7.1). On KITTI tracking training [16], the trackers run with two detectors that differ from the MOT17 detector: YOLOv8n [17] and RT-DETR-L [18]. To study the score problem directly, the published MOT17 scores were also changed before the tracker by four monotone transforms: cubing (s^3), halving ($0.5\,s$), and logit temperatures $T = 2$ and $T = 0.5$, $\sigma(z/T)$. A temperature divides the logit z of each score by T before turning it back into a score: $T = 2$ pulls all scores toward 0.5, and $T = 0.5$ pushes them toward 0 and 1. The trackers kept their published thresholds.

The external systems use the detections released by their authors on MOT17 val-half.

2.2 Evaluation roles and metrics

Every cell has one of two roles (Table 1). Development cells were visible while SCL was designed: MOT17 with four trackers, KITTI with two other detectors, and the score transforms. The predeclared external systems, PD-SORT and Hybrid-SORT, were chosen and declared before they were run, and they were run once with the frozen configuration. For each external tracker, our reproduction is compared with the authors' number and marked EXACT (within 0.2 HOTA) or CLOSE (within 1.0 HOTA, with a documented cause that was not tuned).

Table 2 gives the three tracking metrics, HOTA, multiple object tracking accuracy (MOTA), and identity F1 score (IDF1), and their definitions. They are built from three error counts: false positives (FP), missed objects or false negatives (FN), and identity switches (IDS). They are computed with the TrackEval library (commit 12c8791) and the official ground truth of each benchmark, following MOTChallenge practice [19]. On KITTI tracking, HOTA is the official value averaged over cars and pedestrians.

All comparisons are paired: each tracker runs with and without SCL on the same sequences. Differences are pooled over sequences, and their 95% intervals come from a percentile bootstrap over sequences [22] with 10,000 resamples and seed 42. A difference is called significant when its interval does not include zero. As a simple check that does not depend on any distribution, we also count the sequences where HOTA rose, stayed the same ($|\Delta\text{HOTA}| < 0.01$), or fell.

HOTA, MOTA, and IDF1 are given in percent, and higher values are better. FP, FN, and IDS are counts, and lower values are better. Runtime is given in milliseconds (ms) per frame, and lower values are better.

3 Related Work

SCL is related to four lines of research: the trackers it works with, the confidence of the detectors that feed them, trackers that adapt their own thresholds, and methods that adapt inference while running. They are reviewed below in this order, and Table 3 compares the closest methods with SCL.

Most practical trackers follow the tracking-by-detection design. SORT predicts each box with a Kalman filter and matches the predictions to new detections by overlap with the Hungarian method [4]. ByteTrack added a second matching step that uses low-score detections to continue existing tracks [5], and many later trackers kept this two-stage design. Others, such as OC-SORT [6], use only one score threshold. All these trackers read detector scores through fixed thresholds that were tuned for one detector, usually YOLOX [15]. Seven published trackers of both kinds are used in this paper as hosts and are not changed (Sect. 6).

These fixed thresholds are a weak point, because scores do not mean the same thing for every detector. Detectors of different families, such as YOLOX [15], YOLOv8 [17], YOLOv9 [23], DETR [24], and RT-DETR [18], give different score distributions. Their confidence is not calibrated across models and conditions, and calibration methods for object detection learn a mapping from labelled data [7]. A tracker tuned for one detector therefore cannot assume that another detector's scores mean the same thing, and labels to recalibrate them are rarely available after deployment. Image processing offers a tool that needs no labels: Otsu's method picks a threshold from a histogram so that the two resulting groups are as separate as possible [25]. SCL applies it to the logits (log-odds) of the detector scores of recent frames.

Some trackers already adapt their own thresholds. Adaptive ByteTrack sets the score threshold of ByteTrack from the detections of each frame [9]. AdapTrack adapts the thresholds of its matching step [10]. ACT sets the high and low thresholds of each frame from the first quartile of the scores and the number of candidates [11]. Other recent trackers use the score or the spread of each detection inside the tracker. ConfTrack weights the Kalman update, the Kalman noise, and the matching cost by the score of each box [26]. UTrack measures the spread of the boxes that non-maximum suppression (NMS) merges and uses it as the Kalman noise, so it needs the boxes before suppression [27]. PlugTrack blends the Kalman filter with learned motion predictors through a small network trained with labels [28]. These methods are built into the tracker, use only the current frame

Table 1 Evaluation cells and their role. Development cells were visible while the layer was designed; predeclared cells were run once with the frozen policy.

Data	Detector	Trackers	Role
MOT17 val-half (7 sequences)	published YOLOX-X detections	ByteTrack (2 settings), OC-SORT, BoostTrack, SparseTrack	development
KITTI tracking training (21 sequences)	RT-DETR-L; YOLOv8n	ByteTrack and BoT-SORT (RT-DETR-L); OC-SORT (YOLOv8n)	development (detector transfer)
MOT17 val-half, score transforms	published YOLOX-X, transformed scores	ByteTrack (2 settings), OC-SORT, BoostTrack	development (robustness)
MOT17 val-half	released PD-SORT / Hybrid-SORT detections	PD-SORT, Hybrid-SORT	predeclared external

Table 2 Tracking metrics used in this paper. GT is the number of ground-truth boxes; FP, FN, and IDS are the false positives, missed objects (false negatives), and identity switches, summed over all frames. For IDF1, each ground-truth track is first matched to at most one output track, and IDTP, IDFP, and IDFN are the true-positive, false-positive, and false-negative boxes under this matching. For HOTA, at a localization threshold α, TP_α, FN_α, and FP_α are the matched, missed, and unmatched boxes, and $\mathcal{A}(c)$ is the association accuracy of a matched pair c. HOTA, MOTA, and IDF1 are given in percent.

Metric	Definition	What it measures
MOTA [20]	$\mathrm{MOTA} = 1 - \dfrac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}$	overall error count; reacts strongly to false positives and missed objects
IDF1 [21]	$\mathrm{IDF1} = \dfrac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}$	how long each object keeps the correct identity
HOTA [8]	$\mathrm{HOTA} = \dfrac{1}{19}\sum_\alpha \sqrt{\mathrm{DetA}_\alpha\,\mathrm{AssA}_\alpha}$, $\mathrm{DetA}_\alpha = \dfrac{\lvert\mathrm{TP}_\alpha\rvert}{\lvert\mathrm{TP}_\alpha\rvert + \lvert\mathrm{FN}_\alpha\rvert + \lvert\mathrm{FP}_\alpha\rvert}$, $\mathrm{AssA}_\alpha = \dfrac{1}{\lvert\mathrm{TP}_\alpha\rvert}\sum_{c\in\mathrm{TP}_\alpha}\mathcal{A}(c)$, with $\alpha = 0.05, 0.10, \ldots, 0.95$	balance of detection (DetA) and association (AssA); the main metric of this paper

or each box alone, or need extra trained parts or the inside of the detector. SCL differs in four ways. It uses past frames. It decides for the whole stream whether to act at all, and otherwise keeps the tracker's own settings. It works through a declared host contract on several published trackers without changing them. And it is tested as one frozen configuration under score changes and on external systems.

A wider group of methods adapts inference itself. Dynamic neural networks adapt their computation to each input [12], and Chameleon reprofiles the settings of video analytics pipelines over time [13]. These methods learn or measure a policy for each model. Our earlier work, AC-MOT (a manuscript submitted for publication), is a controller that needs no training and sets the detector operating point (score threshold, NMS threshold, and input resolution) while running, from image and track cues. Such a controller is tied to the detector it was calibrated with, because a change in the detector's score scale does not change the image. SCL reads the scores instead and acts on the tracker's input and thresholds.

In summary, current methods either adapt thresholds or use the score of each box inside one tracker, calibrate scores with labelled data, or learn a policy for one model. What is missing is a layer that needs no training, adjusts itself from the detector's own scores, works with many unchanged trackers, and keeps a well-matched tracker at its own accuracy. SCL is designed to fill this gap. Table 3 summarizes the differences.

4 Method

SCL is a small step that runs once per frame between the detector and the tracker (Fig. 1). It does four things. First, it looks at the detector scores of the last few frames and puts the candidates into three groups: background, uncertain, and confident (Sect. 4.2). Second, it uses these groups to decide whether the detector output is clean or noisy (Sect. 4.3). Third, it acts on this decision: on clean scores it keeps the tracker's

Table 3 Comparison of SCL with recent adaptive methods for tracking (BT: ByteTrack; ReID: re-identification network; NMS: non-maximum suppression).

Method	Year	Adapted	Statistics	Extra trained part	Detector input	Where it works
Adapt. BT [9]	2023	high/low score split	current frame	none	boxes and scores	inside BT
ConfTrack [26]	2024	Kalman update, noise, and cost	score of each box	ReID	boxes and scores	new tracker
AdapTrack [10]	2024	matching thresholds	current frame	ReID	boxes and scores	new tracker
UTrack [27]	2025	Kalman noise	spread of boxes merged by NMS	none	boxes before NMS	added to existing trackers
ACT [11]	2026	high and low thresholds	current frame	none	boxes and scores	inside the tracker
PlugTrack [28]	2026	mix of Kalman and learned motion	track boxes	network trained with labels	boxes	added to two learned predictors
Küppers [7]	2020	detector scores	labelled data	calibration map	boxes and scores	detector output
Chameleon [13]	2018	pipeline configuration	profiling	none	whole pipeline	analytics pipeline
AC-MOT (ours, submitted)	2026	detector operating point	image and track cues	none	sets its settings	before the detector
SCL (this paper)	–	candidates, scores, thresholds	past frames, no labels	none	boxes and scores	between detector and seven unchanged trackers

own settings, and on noisy scores it chooses which candidates reach the tracker, assigns them new scores, and sets the tracker's thresholds. Fourth, in both cases it can pass a weak candidate that clearly continues an existing track (Sect. 4.4). The tracker then runs as usual on what it receives. The detector and the tracker are never changed, and Sect. 4.6 lists every setting of the layer.

4.1 Inputs and outputs

In each frame t, the detector gives a list of candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$: a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class label c_i for each one, down to a lowest score called the emission floor. The layer does not use the score itself but its logit (log-odds), which spreads out scores close to 0 and 1:

$$z_i = \log \frac{s_i}{1 - s_i}. \tag{1}$$

For example, a score of 0.5 gives $z_i = 0$, a score of 0.9 gives $z_i \approx 2.2$, and a score of 0.1 gives $z_i \approx -2.2$. The logistic function $\sigma(z) = 1/(1 + e^{-z})$ turns a logit back into a score. Table 4 lists all symbols of the method.

The tracker, called the host, gives the layer four numbers about itself, the host contract $h = (a, b, \ell, m_0)$. These are simply the settings that its authors published: a is the score a detection needs to be matched to a track in the first step, b is the score it needs to start a new track, ℓ is the lowest score the tracker uses at all, and m_0 is its matching tolerance (one minus the smallest overlap that still counts as a match). A single-stage tracker uses one threshold, so $\ell = \min(a, b)$. A two-stage tracker also uses weaker detections in a second step, so $\ell < \min(a, b)$.

In every frame the layer returns five things: the candidates K_t that the tracker will see, their scores $\tilde{s}$, and the thresholds (a_t, b_t) and tolerance m_t that the tracker will use in this frame:

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t; h\right). \tag{2}$$

Here $Z_{<t}$ are the logits of the earlier frames, $\mathcal{T}_{t-1}$ are the tracks of the previous frame, and g_t is an optional measure of camera motion in frame t. In words, Eq. (2) says that the output for frame t depends only on the logits of earlier frames, the candidates of frame t, the tracks of frame $t-1$, the camera motion, and the tracker's contract. The decision for frame t uses only past frames; frame t is added to the statistics after the decision, and no future frame is ever used.

Four goals guided the design. The layer needs no training and no labels. One frozen configuration must work for every tracker, detector, and dataset. When the tracker already fits the detector, its input must stay the same. And the layer must take little time compared with the detector.

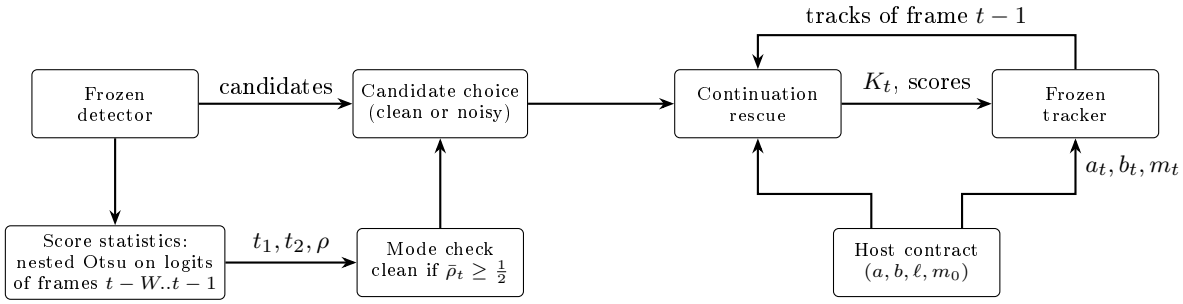


Fig. 1 SCL between a frozen detector and a frozen tracker. The statistics use earlier frames only. The host contract holds the tracker's published settings. The tracker receives the chosen candidates, their scores, and its thresholds for the frame

Table 4 Main symbols of the method.

Symbol	Meaning
t	frame index
D_t	candidates of frame t given by the detector
$\mathbf{x}_i, s_i, c_i$	box, score, and class label of candidate i
z_i	logit (log-odds) of the score s_i, Eq. (1)
$\sigma(\cdot)$	logistic function, turns a logit back into a score
$h = (a, b, \ell, m_0)$	host contract: the tracker's published settings
a, b	score needed to match a track, and to start a new track
ℓ	lowest score the tracker uses
m_0	matching tolerance: one minus the smallest IoU that counts as a match
W	number of past frames used for the statistics (10)
t_1, t_2	first and second Otsu split on the logits
ρ_t, $\bar{\rho}_t$	share of confident candidates in frame t, and its median so far
u_i	rank of candidate i among recent scores, from 0 to 1
K_t, $\tilde{s}$	candidates passed to the tracker, and their scores
a_t, b_t, m_t	thresholds and tolerance used by the tracker in frame t
$\mathcal{T}_{t-1}$	tracks of the previous frame
g_t, r_t	camera motion in frame t, and its ratio to the usual motion
IoU	intersection over union: overlap of two boxes, from 0 to 1

4.2 Self-calibration from detector scores

A detector that suits the scene gives many low scores to background and a separate group of high scores to objects. The layer checks, without labels, how clearly this happens. It collects the logits of the candidates of the last $W = 10$ frames that remain after duplicate removal (Sect. 4.3) and splits them twice with Otsu's method [25], which finds the threshold that best separates a histogram into two groups. The first split, t_1, separates background from foreground. The second split, t_2, divides the foreground into uncertain and confident candidates. The share of confident candidates in the foreground,

$$\rho_t = \frac{|\{i : z_i \geq t_2\}|}{|\{i : z_i \geq t_1\}|}, \tag{3}$$

shows how clearly the detector separates objects from clutter. In words, ρ_t is the number of confident candidates divided by the number of foreground candidates. For example, if 40 candidates are in the foreground and 30 of them are confident, then $\rho_t = 0.75$. A high ρ_t means that most foreground candidates are confident; a low ρ_t means that many of them are uncertain. Because the splits are made on logits, they move with the scores when the whole logit scale is stretched or shifted, for example by a change of temperature, and each group keeps the same candidates.

These groups only make sense if there is a real background group. Some detectors give very few low scores; then the first split falls inside the objects. The layer detects this case, when fewer candidates lie below t_1 than above it, and counts the frame as clean, $\rho_t = 1$. This background check was added after a case seen during development: at the start of some sequences, with about ten candidates per frame, the splits fell at $t_1 \approx 0.6$ and $t_2 \approx 0.93$ in probability terms and gave a wrong noisy decision.

4.3 Clean and noisy modes

The mode decides how much the layer may change. A clean stream means that the detector already separates objects from clutter, so the tracker's own settings are kept. A noisy stream means that many candidates are uncertain, so

the layer takes over the choice of candidates and thresholds.

Choosing the mode. The stream is clean at frame t if the median $\bar{\rho}_t$ of all ρ values so far (ignoring cold frames) is at least one half, and noisy otherwise. Before the first valid split, for example in the first frame, the mode is called cold. Because the median covers the whole stream so far, the mode describes the detector and the scene, and a short drop in confidence, for example during fast camera motion, does not change it.

Cold and clean modes. The tracker keeps its own thresholds. In the clean mode there is one exception: if a threshold is so high that it would reject the confident group, it is lowered to t_2:

$$a_t = \min(a, \sigma(t_2)), \qquad b_t = \min(b, \sigma(t_2)), \quad (4)$$

where σ is the logistic function, which turns a logit back into a score. In words, each threshold stays as the tracker set it, unless it is higher than the score $\sigma(t_2)$ at which the confident group begins; then it is lowered to that score. Scores are passed unchanged, and only cross-class duplicates (one box given two class labels) are removed. When the confident group already lies above the tracker's thresholds, the tracker therefore receives exactly its normal input.

Example. When the detector scores are halved, no score is above 0.5. A tracker with $a = 0.6$ and $b = 0.7$, such as ByteTrack in its official setting (Table 5), then receives no detection that can start a track, so it never creates a track. The Otsu splits are computed from the halved scores themselves, so they move with them. In the clean mode, Eq. (4) then lowers the tracker's thresholds to $\sigma(t_2)$, and the confident candidates reach the tracker again. In the noisy mode, the new scores below place them at or above the thresholds of 0.5. Section 7.3 shows the measured effect.

Noisy mode. Here the layer decides which candidates the tracker may use, and for what. Confident candidates ($z \geq t_2$) may start and continue tracks. Uncertain candidates ($t_1 \leq z < t_2$) may only continue existing tracks. Background candidates ($z < t_1$) are dropped. Each score is replaced by a new one based on its rank u_i among the recent scores (from 0 for the lowest to 1 for the highest),

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \quad (5)$$

and the tracker's matching and birth thresholds are both set to 0.5, $a_t = b_t = 0.5$. In words, the lowest-ranked confident candidate gets 0.5 and the highest-ranked one gets close to 1, while an uncertain candidate gets a score between 0.1 and 0.5 in the same order. Confident candidates thus get scores of 0.5 and above, and uncertain ones get scores below 0.5, so they can only reach a tracker's low-score step. Duplicates are removed with the help of the tracks: a weaker candidate that overlaps a stronger one with an intersection over union (IoU) above one half is removed, unless a different track of the previous frame claims it. This keeps people who stand close together in a crowd.

4.4 Track continuation and camera motion

A tracker ignores every candidate below its lowest threshold, so a track can be lost when its object gets low scores for a few frames. This matters most for single-stage trackers, which have no low-score step. The rescue gives such a candidate a second chance, but only when it clearly continues an existing track. Three conditions must hold: the candidate is in the foreground ($z \geq t_1$, or any candidate for a single-stage tracker when there is no background group); it overlaps a track of the previous frame with an IoU of at least one half; and no other usable candidate covers that track. Also, the tracker must be unable to see the candidate with the thresholds of this frame. The candidate is then passed with its score raised just above the tracker's lowest threshold. For a two-stage tracker, this only applies to candidates below its low threshold ℓ. For a single-stage tracker, it applies to candidates below the lowest threshold used in this frame, so the rescue gives it the continuation step it lacks.

When the camera moves fast, a box moves more between two frames, and a fixed overlap requirement can break a correct match. In noisy frames, and only when a motion measure is available, the matching tolerance therefore grows with the current camera motion; r_t is the current motion divided by its median over past frames.

The motion is the shift between two consecutive grayscale frames at quarter resolution, found by phase correlation [29] and divided by the image diagonal. The tolerance is

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \qquad (6)$$

In words, when the camera moves no faster than usual ($r_t \leq 1$), the tolerance stays at m_0. When the motion is twice its usual value ($r_t = 2$), the gap $1 - m_0$ is halved: with $m_0 = 0.8$, the tolerance becomes 0.9, so a match needs an IoU of only 0.1 instead of 0.2. The tolerance never exceeds 0.95.

4.5 Host contract and adapters

Table 5 lists the contracts used in this paper. Each value is taken from the tracker's published configuration; none is fitted. A small adapter for each tracker family passes the layer's decision to the tracker: the chosen candidates and their scores replace the detector output of the frame, and the new thresholds and tolerance replace the tracker's own for that frame. Any step that the authors apply after tracking, such as interpolation or offline linking, is kept. The rules and their constants are the same for every tracker. A tracker enters only through its four contract values, and the adapter only passes on the decision; it never makes one.

4.6 Fixed constants

Table 6 lists every constant of the layer; the same values are used for every tracker, detector, and dataset. The thresholds t_1 and t_2 are not constants: they are computed again in every frame. Each constant has a simple origin. The overlap of one half for duplicates and rescue follows MOT evaluation, where two boxes with IoU above 0.5 cannot both match different objects. The score ranges of Eq. (5) put the two groups on the common thresholds of 0.5 (matching and birth) and 0.1 (low score). The cap of 0.95 keeps a minimum overlap of 0.05 for every match. The window, the motion history, and the remap memory are default values that were kept fixed in all development stages.

4.7 Per-frame steps and cost

Algorithm 1 gives the steps for one frame. The work per frame is small: the layer sorts the logits of at most W frames for the two Otsu splits, computes overlaps between candidates and with the tracks of the previous frame, and, when motion is used, computes one phase correlation on a quarter-resolution image. With n candidates per frame and k tracks, the cost is $O(Wn \log(Wn) + n^2 + nk)$ plus a fixed image cost. The three terms come from sorting the logits of the window, comparing every pair of candidates, and comparing every candidate with every track. The layer stores only the logits of the last W frames, the per-frame ρ values, the remap memory, and the last 100 motion values.

5 Development of the Method

5.1 Starting point: always-on banding

SCL was developed from an earlier design of the same research program, called always-on banding (AOB). AOB computed the same nested Otsu groups but treated every stream as noisy. In every frame it applied the groups, removed every weaker candidate that overlapped a stronger one with IoU above one half, and assigned all candidates new scores by rank on fixed ranges. It passed no candidate in the first frame. AOB was designed for noisy detector streams, and it is the baseline of this section.

Table 7 compares AOB and SCL with the same strong published trackers on MOT17 [14]. AOB lowered the HOTA of SparseTrack by 4.15 points and that of BoostTrack by 5.89 points online and 5.37 points with BoostTrack's linear and gradient boosting interpolation (GBI), all with intervals below zero. Two causes were found. First, removing overlapping boxes as duplicates deleted people hidden behind others in crowds. Second, the self-calibrated thresholds were stricter than the trackers' own: for SparseTrack and online BoostTrack, FP fell by 1,630 and 1,351, but FN rose by 4,955 and 6,167. On the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack versions identical to the host. Figure 2 shows the same comparison.

Table 5 Host contracts: the published settings of each tracker, as used by the layer.

Tracker (setting)	Type	assoc a	birth b	low ℓ	match m_0	Decision mapped to
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8	thresholds, candidates, scores
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8	same
BoT-SORT (library default)	two-stage	0.25	0.25	0.1	0.8	same
SparseTrack	two-stage	0.6	0.7	0.1	0.85	same
BoostTrack	two-stage	0.6	0.6	0.1	0.7	same
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	same
OC-SORT	single-stage	0.6	0.6	0.6	0.7	same
PD-SORT	single-stage	0.6	0.6	0.6	0.7	same

Table 6 Fixed constants of SCL.

Constant	Value
Statistics window W	10 frames
Mode rule	clean if median $\rho \geq 0.5$ over non-cold frames so far
Background check	$\rho = 1$ if fewer candidates below t_1 than above
Noisy score ranges	primary $[0.5, 1)$, extension $[0.1, 0.5)$
Noisy association and birth thresholds	0.5
Remap memory	20 frames, one every 10 frames
Duplicate and rescue IoU	0.5
Rescued score	lowest used threshold $+0.001$
Motion cue	phase correlation, quarter resolution
Motion history, warm-up	100, 5 frames
Tolerance cap	0.95

What this stage showed. A layer that acts in every frame harms trackers that already fit the detector: strong published trackers lost between 4.15 and 5.89 HOTA points in all three settings. Such a layer cannot be placed safely in front of an unknown tracker. The next steps therefore let the layer decide from the scores whether to act at all.

5.2 Design steps

Each step from AOB to SCL was made to remove one cause found before it, and it was compared with the step before it on development data (Table 8). The steps, in the order in which they were made, were the following.

- *Starting point.* Running AOB inside the new framework showed the same problem: it reduced ByteTrack at floor 0.1 from 67.70 to 56.41 HOTA and OC-SORT from 66.44 to 45.72.
- *Mode gate.* The two causes found in Sect. 5.1 were thresholds stricter than the trackers' own and the removal of hidden people as duplicates. The mode gate adds the mode check, the clean mode that keeps the tracker's settings, and the

Algorithm 1 SCL for frame t. The window holds the logits of frames $t-W$ to $t-1$, so the decision for frame t uses no future frame.

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table 6
Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i ▷ Eq. (1)
if no valid split exists **then** ▷ e.g., first frame
  mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
  $(t_1, t_2) \leftarrow$ nested Otsu on the window ▷ Sect. 4.2
  $\rho_t \leftarrow$ Eq. (3) with the background check
  mode $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy
end if
if mode is cold or clean **then** ▷ Sect. 4.3
  remove cross-class duplicates; keep raw scores
  if clean **then** set (a_t, b_t) by Eq. (4)
  end if
else ▷ noisy mode
  remove duplicates with track context
  drop $z < t_1$; remap scores by Eq. (5)
  $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold) ▷ Sect. 4.4
$m_t \leftarrow$ Eq. (6) if noisy, else m_0
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits left after duplicate removal to the window
add g_t to the motion history
store the tracks of frame t after the tracker update

Table 7 Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections). Host: published tracker alone (our reproduction). Δ: tracker with the earlier design AOB or with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero.

Metric	Host	Δ AOB	Δ SCL
SparseTrack [30]			
HOTA	68.88	**−4.15**	+0.06
		[−5.54, −1.66]	[−0.01, +0.25]
MOTA	77.85	**−6.14**	+0.08
		[−8.10, −2.74]	[−0.03, +0.30]
IDF1	81.97	**−4.49**	+0.16
		[−6.19, −1.96]	[−0.02, +0.70]
BoostTrack, online [31]			
HOTA	68.49	**−5.89**	0 (identical)
		[−7.76, −2.87]	
MOTA	75.50	**−8.87**	0 (identical)
		[−12.37, −7.14]	
IDF1	81.41	**−6.22**	0 (identical)
		[−9.33, −2.85]	
BoostTrack, with GBI [31]			
HOTA	71.72	**−5.37**	0 (identical)
		[−7.30, −2.75]	
MOTA	81.03	**−8.73**	0 (identical)
		[−12.59, −6.50]	
IDF1	84.16	**−5.65**	0 (identical)
		[−8.96, −2.77]	

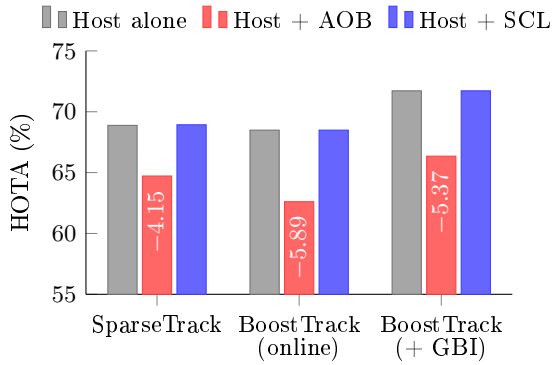


Fig. 2 HOTA on MOT17 val-half of strong published trackers (SparseTrack and BoostTrack in two settings) alone, with AOB, and with SCL. The labels give the change caused by AOB

crowd-safe duplicate rule. It restored SparseTrack from 64.72 to 68.93 and BoostTrack from 62.61 to 68.37.

- *Whole-stream mode.* The mode still changed in the middle of some streams. Choosing it from the whole stream removed these changes and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071.
- *Continuation rescue.* Single-stage trackers still lost tracks whose objects got low scores for a few frames. The rescue added 0.2 to 0.4 points for OC-SORT.
- *Background check.* A stream with few low-score candidates gave a wrong noisy decision (Sect. 4.2). The background check completed SCL: it returned ByteTrack and BoostTrack to output identical to the host and raised OC-SORT to 67.04 and 66.91 at the two floors.

Four other versions were tested and rejected, because they lowered HOTA, added FP, or let false tracks through on a noisy stream.

What this stage showed. After these steps, the layer kept the strong two-stage trackers at their own accuracy and improved the single-stage OC-SORT. This is the behavior that the frozen configuration was then tested for on new trackers, detectors, and data (Sect. 7).

5.3 Freezing and testing

The final configuration was frozen before any external test. It was saved in a version-control tag and protected by a lock file that holds cryptographic hashes of the layer, its configuration, and the evaluation scripts. Every later run checked the lock before running. Sixty-one automated tests check that the layer uses no future data, that its state resets between sequences, that clean streams pass through unchanged, that the output does not depend on detector, tracker, or sequence names, that affine score changes leave the groups unchanged where designed, and that the lock is intact.

6 Experimental Setup

The experiments test three expectations from the design. A tracker that already fits its detector should keep its accuracy. A tracker that does not fit, because it has no continuation step or because the detector or its score scale changed, should gain. Both should hold on trackers and data that were not seen during design.

Seven published trackers serve as hosts: ByteTrack [5], BoT-SORT [32], OC-SORT [6], SparseTrack [30], BoostTrack [31], PD-SORT [33], and Hybrid-SORT [34]. Each host uses the authors' code and published configuration, and its contract is given in Table 5.

Table 8 Design steps from AOB to SCL on development data (HOTA, %), and versions that were tested and rejected. The floor of ByteTrack and OC-SORT is given in parentheses.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)
Rejected version		*Evidence*
no candidates at frame 1		−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage		FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames		ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band		KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

In every comparison the baseline is the host alone: the published tracker, run by us with its published configuration and the same detections. The second run adds the frozen SCL and changes nothing else. Both runs use the same environment. Where the authors report their own numbers, these are listed next to our reproduction of the host.

7 Results and Analysis

Figure 3 shows the HOTA difference between each host with SCL and the host alone for every tested cell. The next subsections give the baseline comparison for each part of the evaluation, in the order of the three expectations of Sect. 6: well-matched trackers, trackers that do not fit their detector, and systems not seen during design.

7.1 Results across trackers

Table 9 compares the development trackers with their hosts on MOT17. The two-stage trackers with well-matched detections stayed at the accuracy of their hosts. BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave identical output. ByteTrack at floor 0.01 changed by −0.014 HOTA and SparseTrack by +0.055. This is the self-limiting behavior of the clean mode (Sect. 4.3), which the earlier design did not have (Table 7).

The single-stage OC-SORT improved at both emission floors. Its HOTA rose by 0.61 points ([+0.36, +1.24]) at floor 0.01 and by 0.46 points ([+0.27, +1.09]) at floor 0.1, and its MOTA by

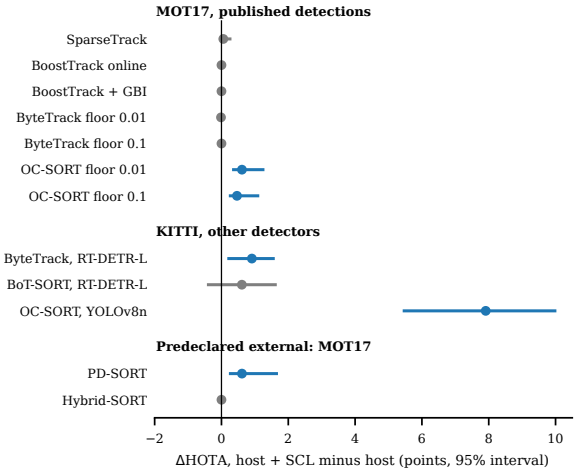


Fig. 3 Difference in HOTA (host with SCL minus host alone) with 95% paired-bootstrap interval for every tested cell. Blue: interval above zero; gray: interval contains zero, or identical output

1.27 and 1.26 points. HOTA was higher on 7 of 7 sequences in both settings (Fig. 4). The gain comes from the continuation rescue. OC-SORT drops every detection below its single threshold, and the layer passes it the foreground candidates that continue a track that would otherwise stay unmatched.

7.2 Results with other detectors

Table 10 reports the KITTI results with the two other detectors. With the noisy RT-DETR-L detector, the two-stage trackers gained in MOTA and IDF1. ByteTrack gained 5.98 MOTA points ([+1.16, +9.94]), 3.87 IDF1 points, and

Table 9 Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections). Paper: number reported by the tracker's authors. Host: published tracker alone (our reproduction). Δ: host with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. The floor is the lowest detector score emitted.

Metric	Paper	Host	Δ [95% CI]
SparseTrack [30]			
HOTA	69.2	68.88	+0.06 [−0.01, +0.25]
MOTA	76.8	77.85	+0.08 [−0.03, +0.30]
IDF1	81.4	81.97	+0.16 [−0.02, +0.70]
BoostTrack, online [31]			
HOTA	68.37	68.49	0 (identical)
MOTA	75.56	75.50	0 (identical)
IDF1	81.35	81.41	0 (identical)
BoostTrack, + GBI [31]			
HOTA	71.33	71.72	0 (identical)
MOTA	80.55	81.03	0 (identical)
IDF1	83.84	84.16	0 (identical)
ByteTrack, floor 0.01 [5]			
HOTA	–	67.70	−0.014 [−0.058, +0.009]
MOTA	76.6	77.60	+0.06 [−0.15, +0.34]
IDF1	79.3	79.47	−0.031 [−0.135, +0.035]
ByteTrack, floor 0.1 [5]			
HOTA	–	67.70	0 (identical)
MOTA	76.6	77.60	0 (identical)
IDF1	79.3	79.47	0 (identical)
OC-SORT, floor 0.01 [6]			
HOTA	66.5	66.43	**+0.61 [+0.36, +1.24]**
MOTA	74.9	74.67	**+1.27 [+0.17, +3.01]**
IDF1	77.7	78.05	+0.66 [−0.25, +1.60]
OC-SORT, floor 0.1 [6]			
HOTA	66.5	66.44	**+0.46 [+0.27, +1.09]**
MOTA	74.9	74.67	**+1.26 [+0.37, +3.34]**
IDF1	77.7	78.05	**+0.287 [+0.003, +0.838]**

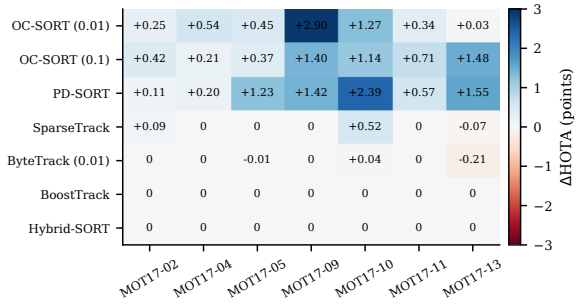


Fig. 4 Per-sequence HOTA differences (host with SCL minus host alone) on MOT17 val-half for the development and predeclared trackers

0.91 HOTA points ([+0.22, +1.55]). BoT-SORT gained 7.86 MOTA points ([+2.21, +13.44]) and 3.10 IDF1 points. With YOLOv8n, the single-stage OC-SORT gained 7.91 HOTA points ([+5.47, +9.98]), 9.40 MOTA points, and 9.60 IDF1 points.

Table 10 Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: published tracker alone with the same detections. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. BoT-SORT uses 20 of the 21 sequences in both arms: sequence 0020 is excluded because of a numerical error in the BoT-SORT Kalman update.

Metric	Host	Δ [95% CI]
ByteTrack [5], *RT-DETR-L*		
HOTA	50.73	**+0.91 [+0.22, +1.55]**
MOTA	47.27	**+5.98 [+1.16, +9.94]**
IDF1	64.98	**+3.87 [+1.67, +5.06]**
BoT-SORT [32], *RT-DETR-L*		
HOTA	53.91	+0.61 [−0.39, +1.61]
MOTA	48.25	**+7.86 [+2.21, +13.44]**
IDF1	67.74	**+3.10 [+1.00, +4.57]**
OC-SORT [6], *YOLOv8n*		
HOTA	36.27	**+7.91 [+5.47, +9.98]**
MOTA	33.99	**+9.40 [+1.10, +12.30]**
IDF1	50.46	**+9.60 [+5.61, +12.16]**

7.3 Results under shifted detector scores

Table 11 and Fig. 5 show the trackers under the four score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA, because no detection reached their thresholds. With the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60. BoostTrack rose from 61.33 to 64.84 under cubing. With temperature $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. The library setting of ByteTrack, whose low thresholds already accept the changed scores, stayed within 0.14 points in all four transforms. Over the 14 conditions, the layer raised HOTA significantly in 7 and lowered it significantly in none.

7.4 Results on external trackers

Table 12 gives the two predeclared external systems. PD-SORT [33] was reproduced exactly as in the authors' released output. With SCL, its HOTA rose by 0.61 points ([+0.27, +1.65]), its MOTA by 1.13 points ([+0.27, +3.03]), and its IDF1 by 0.82 points ([+0.43, +1.94]), with higher HOTA on 7 of 7 sequences (Fig. 4). FN fell by 1,291, and FP rose by 681. PD-SORT is a single-stage tracker, and its gain comes from the same

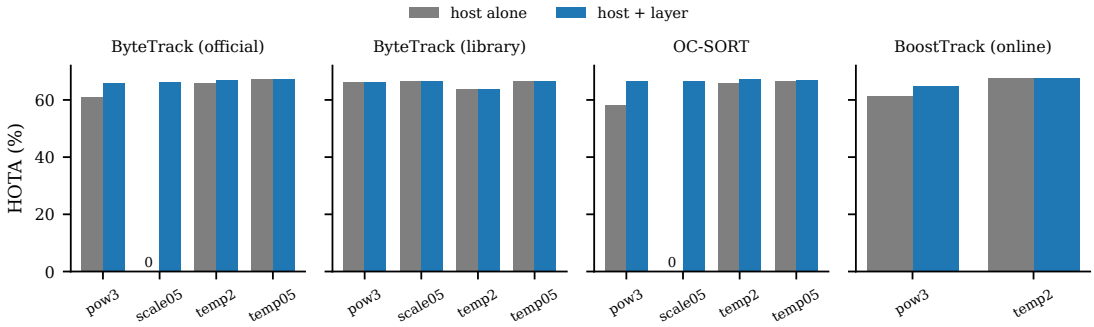


Fig. 5 Shifted detector scores on MOT17 val-half: HOTA of each published tracker with its published thresholds, alone and with SCL, after four monotone transforms of the detector scores (pow3: s^3; scale05: $0.5\,s$; temp2 and temp05: $T = 2$ and $T = 0.5$)

Table 11 Baseline comparison under shifted detector scores on MOT17 val-half (HOTA). The detector scores are transformed before the tracker, whose published thresholds stay fixed; T is a logit temperature, $\sigma(z/T)$. Host: published tracker alone. Δ: with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero. Emission floor 0.01 (BoostTrack 0.1).

Transform	Host	+ SCL	ΔHOTA
ByteTrack, official setting [5]			
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**
$T = 0.5$	67.30	67.29	−0.009 [−0.051, +0.005]
ByteTrack, library setting [5]			
s^3	66.31	66.31	−0.006 [−0.384, +0.383]
$0.5\,s$	66.61	66.48	−0.14 [−0.66, +0.14]
$T = 2$	63.97	63.97	0 (identical)
$T = 0.5$	66.53	66.52	−0.006 [−0.029, +0.001]
OC-SORT [6]			
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**
$T = 0.5$	66.61	67.06	+0.45 [−0.01, +0.97]
BoostTrack, online [31]			
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**
$T = 2$	67.59	67.59	0 (identical)

Table 12 Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL. Paper: number reported by the authors (for PD-SORT, their released output). Host: published tracker alone (our reproduction). Reproduction class against the authors' number (EXACT: $|\Delta\text{HOTA}| \leq 0.2$; CLOSE: ≤ 1.0 with a documented, untuned cause). Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero.

Metric	Paper	Host	Δ [95% CI]
PD-SORT [33], *EXACT*			
HOTA	68.01	68.01	**+0.61 [+0.27, +1.65]**
MOTA	75.19	75.19	**+1.13 [+0.27, +3.03]**
IDF1	81.03	81.03	**+0.82 [+0.43, +1.94]**
Hybrid-SORT [34], *CLOSE*			
HOTA	67.1	66.70	0 (identical)
MOTA	75.8	75.52	0 (identical)
IDF1	78.0	77.56	0 (identical)

step as for OC-SORT; unlike OC-SORT, it was not seen during design. Hybrid-SORT [34], a two-stage tracker, gave identical output, as predicted before the run.

7.5 Running time

Running time was measured after the freeze on one cloud machine with a 4-vCPU Intel Xeon processor at 2.10 GHz, 15 GB of memory, and no graphics processing unit (GPU). A live YOLOv8n detector (32-bit floats, 736 pixels, batch 1) feeds ByteTrack on three KITTI tracking sequences (0001, 0009, and 0019). Both runs use the same frames one after the other, and the first ten frames of each sequence are excluded, which leaves 2,279 timed frames per run. The time of the layer includes the motion cue, the per-frame step, and saving the tracker output.

Table 13 and Fig. 6 compare the time per frame of the host pipeline with and without SCL. SCL took 2.95 ms per frame on average and 4.39 ms at the 95th percentile, including the phase-correlation motion cue. The total time rose from 57.85 to 60.44 ms per frame, by 4.5%. The tracker became faster, from 2.04 to 1.43 ms per frame, because it received fewer candidates. The detector takes most of each frame, about 42 ms, so the layer adds only a small part of the total time.

Table 13 Baseline comparison of the per-frame time on a 4-vCPU Intel Xeon (2.10 GHz, no GPU) with a live YOLOv8n detector (736 pixels) and ByteTrack on three KITTI sequences (2,279 timed frames per arm). Host alone: the same pipeline without SCL. Mean / 95th percentile in milliseconds.

Stage	Host alone	Host + SCL
Frame decode	13.88 / 16.21	13.88 / 16.51
Detector (YOLOv8n, 736 px)	41.91 / 58.41	42.17 / 59.43
SCL	–	2.95 / 4.39
Tracker (ByteTrack)	2.04 / 3.57	1.43 / 2.64
End to end	57.85 / 76.35	60.44 / 80.78

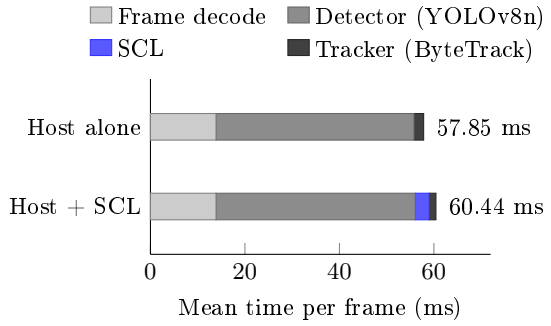


Fig. 6 Mean per-frame time by stage for the host pipeline alone and with SCL (4-vCPU Intel Xeon, no GPU). The labels give the measured end-to-end time

7.6 Analysis

When the layer acts. The effect depends on two properties of the pipeline, not on the dataset: whether the tracker has a continuation step, and whether the detector's scores match the tracker's thresholds. On MOT17 with well-matched detections, the two-stage trackers BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT produced output identical to their hosts, while the single-stage OC-SORT and PD-SORT gained HOTA on every sequence. When the stream fits the tracker, the layer stays out of the way, and when the tracker lacks a step, the layer adds it.

Self-limiting by design. In the clean mode the layer keeps the tracker's thresholds and scores, so a tracker with a low-score step and well-matched detections receives its normal input. This is the main difference from the earlier design AOB, which lowered the HOTA of the same strong trackers by 4.15 to 5.89 points (Table 7). The layer therefore does not need to know in advance whether a tracker needs help; it reads the answer from the scores.

A continuation step for single-stage trackers. Single-stage trackers drop every detection below one threshold, so a track can be lost when its object receives low scores for a few frames. The continuation rescue considers only candidates that overlap a track of the previous frame that no usable candidate covers, so it targets exactly these gaps. OC-SORT and the external PD-SORT gained HOTA on 7 of 7 MOT17 sequences through this step.

Detector replacement. When a detector is replaced or its score scale shifts, the fixed thresholds of a tracker no longer match the scores. The layer adjusts them from the scores without labels. Under halved scores it restored two trackers from 0 to about 66 HOTA, and with a different detector it raised the MOTA of two two-stage trackers by 6.0 and 7.9 points. This is the case the layer was built for: a detector that is swapped, retrained, or quantized while the tracker stays the same.

Relation to detector-side control. Our ACMOT controller acts before the detector and chooses its operating point from the scene. SCL acts after the detector and adapts the tracker's input to the scores that the detector produces. The two layers work on different parts of the pipeline, so they could be used together; combining them is left for future work.

Real-time use. The layer needs no training, no GPU, and no access to the inside of the detector, and it needs only four contract values from the tracker's configuration. At run time it reads the candidate list, one small grayscale image for the motion cue, and the tracks of the previous frame. On a CPU it takes 2.95 ms per frame, adds 4.5% to the total time, and makes the tracker itself faster. SCL therefore adds very little delay and is suitable for real-time use. New trackers are added through their published contract values and an adapter, without changes to the rules.

8 Conclusion

This paper presented SCL, a control layer that needs no training, adjusts itself from a detector's scores, and adapts a frozen tracker through a declared host contract. The layer splits the logits of recent frames with nested Otsu thresholds and decides whether the stream is clean or noisy. On clean streams it keeps the tracker's settings; on noisy streams it chooses the candidates, assigns

them new scores, and sets the thresholds. A continuation rescue gives single-stage trackers the step they lack.

One frozen configuration, developed step by step from AOB, was tested on seven published trackers, four detection sources, and two datasets. It improved single-stage trackers and trackers whose detector or score scale had changed, kept well-matched two-stage trackers at their own accuracy, and took 2.95 ms per frame on a CPU. These results support placing a self-calibrating layer between the detector and the tracker in systems where the detector changes while the tracker stays the same.

Statements and Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval and consent to participate. Not applicable. The study uses only public benchmark datasets and involves no human participants or animals.

Data availability. The datasets analysed in this study are publicly available: MOT17 (https://motchallenge.net) and KITTI tracking (https://www.cvlibs.net/datasets/kitti). The detections released by the authors of the evaluated trackers were obtained from their public repositories. The per-sequence results and bootstrap outputs generated in this study are available from the corresponding author on reasonable request.

Code availability. The code of the layer, its frozen configuration, the tracker adapters, and the evaluation scripts are available from the corresponding author on reasonable request.

Author contributions. Ahmed Gouda Ismail: conceptualization, methodology, software, investigation, formal analysis, and writing of the original draft. Mohamed S. Mohamed and Tarek Ahmed Mahmoud: supervision, and review and editing of the manuscript. All authors read and approved the final manuscript.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Wu, X., Li, W., Hong, D., Tao, R., Du, Q.: Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. IEEE Geoscience and Remote Sensing Magazine **10**(1), 91–124 (2022) https://doi.org/10.1109/mgrs.2021.3115137

[3] Lu, Y.-R., Sun, D.: Development of Real-Time Unmanned Aerial Vehicle Urban Object Detection System with Federated Learning. Journal of Aerospace Information Systems **21**(7), 547–553 (2024) https://doi.org/10.2514/1.i011378

[4] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[5] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[6] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[7] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https:

//doi.org/10.1109/cvprw50498.2020.00171

[8] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[9] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[10] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[11] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/electronics15174047

[12] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[13] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: Scalable Adaptation of Video Analytics. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, pp. 253–266 (2018). https://doi.org/10.1145/3230543.3230574

[14] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016). https://doi.org/10.48550/arXiv.1603.00831

[15] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021). https://doi.org/10.48550/arXiv.2107.08430

[16] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[17] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[18] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-time Object Detection. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[19] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[20] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing **2008**, 1–10 (2008) https://doi.org/10.1155/2008/246309

[21] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-target, Multi-camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[22] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap, (1993). https://doi.org/10.1007/978-1-4899-4541-9

[23] Wang, C.-Y., Yeh, I.-H., Liao, H.-Y.M.:

YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. In: Computer Vision – ECCV 2024. Lecture Notes in Computer Science, pp. 1–21 (2024). https://doi.org/10.1007/978-3-031-72751-1_1

[24] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-End Object Detection with Transformers. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, pp. 213–229 (2020). https://doi.org/10.1007/978-3-030-58452-8_13

[25] Otsu, N.: A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics **9**(1), 62–66 (1979) https://doi.org/10.1109/tsmc.1979.4310076

[26] Jung, H., Kang, S., Kim, T., Kim, H.: ConfTrack: Kalman Filter-based Multi-Person Tracking by Utilizing Confidence Score of Detection Box. In: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 6569–6578 (2024). https://doi.org/10.1109/wacv57701.2024.00645

[27] Solano-Carrillo, E., Sattler, F., Alex, A., Klein, A., Costa, B.P., Rodriguez, A.B., Stoppe, J.: UTrack: Multi-object Tracking with Uncertain Detections. In: Computer Vision – ECCV 2024 Workshops. Lecture Notes in Computer Science, pp. 219–236. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-91585-7_14

[28] Kim, S., Lee, S., Cho, M.: PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 5709–5716 (2026). https://doi.org/10.1609/aaai.v40i7.37491

[29] Reddy, B.S., Chatterji, B.N.: An FFT-based technique for translation, rotation, and scale-invariant image registration. IEEE Transactions on Image Processing **5**(8), 1266–1271 (1996) https://doi.org/10.1109/83.506761

[30] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[31] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: boosting the similarity measure and detection confidence for improved multiple object tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[32] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022). https://doi.org/10.48550/arXiv.2206.14651

[33] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[34] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471